

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 8
з курсу “Операційні системи реального часу”
на тему: “Управління багатозадачною системою за допомогою ІЧ-
пульта”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Мета роботи. Навчитися приймати та обробляти сигнали від інфрачервоного пульта дистанційного керування та реалізувати вибір задач або режимів роботи системи за допомогою ІЧ-пульта.

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>
```

```
/* Pico SDK */

#include "pico/stdlib.h"

#include "hardware/adc.h"

#include "hardware/gpio.h"

#include "hardware/pio.h"
```

```
/* FreeRTOS */

#include "FreeRTOS.h"

#include "task.h"

#include "queue.h"

#include "semphr.h"
```

```
/* Waveshare Libs (OLED) */

#include "DEV_Config.h"

#include "GUI_Paint.h"

#include "OLED_1in5.h"
```

```
/* Sensor Libs */

#include "SHTC3.h"

#include "SGP40.h"

#include "QMI8658.h"
```

```
/* IR Receiver */

#include "nec_receive.h"
```

```

/* Defines */

#define PIO_NUM (pio0)

#define IR_RX_PIN (15)

#define BUTTON_PIN 3

#define PIN_POT 26

#define PIN_LDR 27

#define PIN_MIC 28


/* RTOS Handles */

static SemaphoreHandle_t i2c_mutex;

static QueueHandle_t xQueueButton;

static QueueHandle_t xQueueSHTC3;

static QueueHandle_t xQueueSGP40;

static QueueHandle_t xQueueQMI8658;

static QueueHandle_t xQueuePot;

static QueueHandle_t xQueueLight;

static QueueHandle_t xQueueSound;

static QueueHandle_t xQueueIR;


/* Global OLED Buffer */

static UBYTE *BlackImage;


/* Global PIO and State Machine for IR */

static PIO pio;

static int rx_sm;


/* Data Structures for Queues */

typedef struct {

    float temp;

    float hum;

```

```
} SHTC3_data_t;
```

```
typedef struct {  
    float acc[3];  
    float gyro[3];  
} QMI8658_data_t;
```

```
typedef struct {  
    uint8_t address;  
    uint8_t data;  
} IR_command_t;
```

```
typedef enum {  
    SENSOR_SHTC3 = 0,  
    SENSOR_SGP40,  
    SENSOR_QMI8658,  
    SENSOR_POT,  
    SENSOR_LIGHT,  
    SENSOR_SOUND,  
    SENSOR_COUNT  
} sensor_display_t;
```

```
/**
```

```
 * @brief Task that polls button with debounce logic.
```

```
*/
```

```
void vButtonTask(void *pvParameters) {  
    gpio_init(BUTTON_PIN);  
    gpio_set_dir(BUTTON_PIN, GPIO_IN);  
    gpio_pull_up(BUTTON_PIN);
```

```

bool last_state = true; // true = released (due to pull-up)

while (1) {

    bool current_state = gpio_get(BUTTON_PIN);

    // If button is pressed (false) and was previously released (true)
    if (current_state == false && last_state == true) {

        // Debounce delay
        vTaskDelay(pdMS_TO_TICKS(50));

        current_state = gpio_get(BUTTON_PIN);

        // If still pressed, send event
        if (current_state == false) {
            bool pressed = true;

            xQueueSend(xQueueButton, &pressed, 0);
        }
    }

    last_state = current_state;

    vTaskDelay(pdMS_TO_TICKS(50));
}

/**
 * @brief Task for reading temperature/humidity sensor SHTC3.
 */

void vSHTC3Task(void *pvParameters) {

    SHTC3_data_t data;

    while (1) {

```

```

// Take mutex for I2C access

if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {

    SHTC3_Measurement(&data.temp, &data.hum);

    xSemaphoreGive(i2c_mutex);

    // Send data to queue

    xQueueSend(xQueueSHTC3, &data, 0);

}

vTaskDelay(pdMS_TO_TICKS(100)); // Read every 100 ms

}

}

/**
 * @brief Task for reading air quality sensor SGP40.
 */

void vSGP40Task(void *pvParameters) {

    uint32_t voc_index;

    while (1) {

        if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {

            // Use T=25C, H=50% as in example

            voc_index = SGP40_MeasureVOC(25, 50);

            xSemaphoreGive(i2c_mutex);

            xQueueSend(xQueueSGP40, &voc_index, 0);

        }

        vTaskDelay(pdMS_TO_TICKS(100)); // Read every 100 ms

    }

```

```
}
```

```
/**
```

```
 * @brief Task for reading accelerometer/gyroscope QMI8658.
```

```
 */
```

```
void vQMI8658Task(void *pvParameters) {
```

```
    QMI8658_data_t data;
```

```
    unsigned int tim_count = 0;
```

```
    while (1) {
```

```
        if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {
```

```
            QMI8658_read_xyz(data.acc, data.gyro, &tim_count);
```

```
            xSemaphoreGive(i2c_mutex);
```

```
            xQueueSend(xQueueQMI8658, &data, 0);
```

```
        }
```

```
        vTaskDelay(pdMS_TO_TICKS(100)); // Read every 100 ms
```

```
    }
```

```
}
```

```
/**
```

```
 * @brief Task for reading potentiometer from ADC channel 0 (GPIO26).
```

```
 */
```

```
void vPotTask(void *pvParameters) {
```

```
    while (1) {
```

```
        adc_select_input(0); // GPIO26 -> ADC channel 0
```

```
        sleep_us(5); // Allow multiplexer to settle
```

```
        uint16_t value = adc_read();
```

```

xQueueSend(xQueuePot, &value, 0);

vTaskDelay(pdMS_TO_TICKS(50)); // Read every 50 ms
}
}

/**
 * @brief Task for reading photoresistor from ADC channel 1 (GPIO27).
 */
void vLightTask(void *pvParameters) {
    while (1) {
        adc_select_input(1); // GPIO27 -> ADC channel 1
        sleep_us(5); // Allow multiplexer to settle
        uint16_t value = adc_read();

        xQueueSend(xQueueLight, &value, 0);

        vTaskDelay(pdMS_TO_TICKS(50)); // Read every 50 ms
    }
}

/**
 * @brief Task for reading sound sensor from ADC channel 2 (GPIO28).
 */
void vSoundTask(void *pvParameters) {
    while (1) {
        adc_select_input(2); // GPIO28 -> ADC channel 2
        sleep_us(5); // Allow multiplexer to settle
        uint16_t value = adc_read();

```



```

xQueueSend(xQueueSound, &value, 0);

vTaskDelay(pdMS_TO_TICKS(50)); // Read every 50 ms
}
}

/**
 * @brief Task for receiving and decoding IR remote signals.
 */
void vIRTask(void *pvParameters) {
    uint8_t rx_address, rx_data;

    while (1) {
        // Poll for IR frames

        while (!pio_sm_is_rx_fifo_empty(pio, rx_sm)) {
            uint32_t rx_frame = pio_sm_get(pio, rx_sm);

            if (nec_decode_frame(rx_frame, &rx_address, &rx_data)) {
                // Only send commands for address 0x00 with data 0x07 or 0x15
                if (rx_address == 0x00 && (rx_data == 0x07 || rx_data == 0x15)) {
                    IR_command_t cmd;

                    cmd.address = rx_address;

                    cmd.data = rx_data;

                    xQueueSend(xQueueIR, &cmd, 0);

                    printf("IR command: %02x, %02x\r\n", rx_address, rx_data);
                }
            }
        }
    }

    vTaskDelay(pdMS_TO_TICKS(10)); // Poll every 10 ms
}

```

```

}

}

/**
 * @brief Task for displaying sensor data on OLED with sequential cycling.
 */

void vDisplayTask(void *pvParameters) {

    static sensor_display_t current_sensor = SENSOR_SHTC3;

    // Local variables for storing latest data
    SHTC3_data_t shtc3_data = {0};
    uint32_t voc_data = 0;
    QMI8658_data_t qmi_data = {{0}, {0}};
    uint16_t pot_data = 0;
    uint16_t light_data = 0;
    uint16_t sound_data = 0;
    IR_command_t ir_cmd;
    char str[40];

    while (1) {
        // Check for IR commands (non-blocking)
        if (xQueueReceive(xQueueIR, &ir_cmd, 0) == pdPASS) {
            if (ir_cmd.address == 0x00) {
                if (ir_cmd.data == 0x07) {
                    // Minus button: cycle to previous sensor
                    current_sensor = (sensor_display_t)((current_sensor - 1 + SENSOR_COUNT) % SENSOR_COUNT);
                } else if (ir_cmd.data == 0x15) {
                    // Plus button: cycle to next sensor
                    current_sensor = (sensor_display_t)((current_sensor + 1) % SENSOR_COUNT);
                }
            }
        }
    }
}

```

```

    }
}

// Receive data from all sensor queues (non-blocking)
xQueueReceive(xQueueSHTC3, &shtc3_data, 0);
xQueueReceive(xQueueSGP40, &voc_data, 0);
xQueueReceive(xQueueQMI8658, &qmi_data, 0);
xQueueReceive(xQueuePot, &pot_data, 0);
xQueueReceive(xQueueLight, &light_data, 0);
xQueueReceive(xQueueSound, &sound_data, 0);

// Display current sensor data
Paint_SelectImage(BlackImage);
Paint_Clear(BLACK);

switch (current_sensor) {
    case SENSOR_SHTC3:
        sprintf(str, "SHTC3");
        Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);
        sprintf(str, "T: %.2f C", shtc3_data.temp);
        Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);
        sprintf(str, "H: %.2f %%", shtc3_data.hum);
        Paint_DrawString_EN(5, 35, str, &Font12, WHITE, BLACK);
        break;

    case SENSOR_SGP40:
        sprintf(str, "SGP40");
        Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);
        sprintf(str, "VOC: %u", (unsigned int)voc_data);
        Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);

```

```
break;
```

```
case SENSOR_QMI8658:
```

```
    sprintf(str, "QMI8658");  
    Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);  
    sprintf(str, "AccX: %.2f", qmi_data.acc[0]);  
    Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);  
    sprintf(str, "AccY: %.2f", qmi_data.acc[1]);  
    Paint_DrawString_EN(5, 35, str, &Font12, WHITE, BLACK);  
    sprintf(str, "AccZ: %.2f", qmi_data.acc[2]);  
    Paint_DrawString_EN(5, 50, str, &Font12, WHITE, BLACK);  
    sprintf(str, "GyroX: %.2f", qmi_data.gyro[0]);  
    Paint_DrawString_EN(5, 65, str, &Font12, WHITE, BLACK);  
    sprintf(str, "GyroY: %.2f", qmi_data.gyro[1]);  
    Paint_DrawString_EN(5, 80, str, &Font12, WHITE, BLACK);  
    sprintf(str, "GyroZ: %.2f", qmi_data.gyro[2]);  
    Paint_DrawString_EN(5, 95, str, &Font12, WHITE, BLACK);  
    break;
```

```
case SENSOR_POT:
```

```
    sprintf(str, "Potentiometer");  
    Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);  
    sprintf(str, "Value: %u", pot_data);  
    Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);  
    sprintf(str, "Percent: %.1f%%", (pot_data * 100.0f / 4095.0f));  
    Paint_DrawString_EN(5, 35, str, &Font12, WHITE, BLACK);  
    break;
```

```
case SENSOR_LIGHT:
```

```
    sprintf(str, "Photoresistor");
```

```

    Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);

    sprintf(str, "Value: %u", light_data);

    Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);

    sprintf(str, "Percent: %.1f%%", (light_data * 100.0f / 4095.0f));

    Paint_DrawString_EN(5, 35, str, &Font12, WHITE, BLACK);

    break;

case SENSOR_SOUND:

    sprintf(str, "Sound Sensor");

    Paint_DrawString_EN(5, 5, str, &Font12, WHITE, BLACK);

    sprintf(str, "Value: %u", sound_data);

    Paint_DrawString_EN(5, 20, str, &Font12, WHITE, BLACK);

    sprintf(str, "Percent: %.1f%%", (sound_data * 100.0f / 4095.0f));

    Paint_DrawString_EN(5, 35, str, &Font12, WHITE, BLACK);

    break;

default:

    break;
}

OLED_1in5_Display(BlackImage);

// Update display every 200 ms
vTaskDelay(pdMS_TO_TICKS(200));
}
}

int main(void) {
    // Initialize system

    stdio_init_all();

```

```

if (DEV_Module_Init() != 0) {
    printf("DEV_Module_Init failed\r\n");
    while(1);
}
printf("DEV_Module_Init OK\r\n");

// Initialize OLED
OLED_1in5_Init();
OLED_1in5_Clear(0);

// Create display buffer
UWORD Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH / 2) : (OLED_1in5_WIDTH / 2
+ 1)) * OLED_1in5_HEIGHT;

if ((BlackImage = (UBYTE *)malloc(Imagesize)) == NULL) {
    printf("Failed to apply for black memory...\r\n");
    while(1);
}
Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0, WHITE);
Paint_SetScale(16);
Paint_Clear(BLACK);
OLED_1in5_Display(BlackImage);

// Initialize PIO for IR receiver
pio = PIO_NUM;
rx_sm = nec_rx_init(pio, IR_RX_PIN);
if (rx_sm == -1) {
    printf("could not configure PIO\n");
    while(1);
}
printf("IR RX initialized\r\n");

```

```

// Initialize ADC hardware

adc_init();

adc_gpio_init(PIN_POT); // GPIO26 -> ADC channel 0
adc_gpio_init(PIN_LDR); // GPIO27 -> ADC channel 1
adc_gpio_init(PIN_MIC); // GPIO28 -> ADC channel 2
printf("ADC initialized\r\n");


// Initialize I2C mutex
i2c_mutex = xSemaphoreCreateMutex();

if (i2c_mutex == NULL) {
    printf("Failed to create I2C mutex\r\n");
    while(1);
}


// Initialize I2C sensors (with mutex protection)
xSemaphoreTake(i2c_mutex, portMAX_DELAY);
SHTC3_Init();
SGP40_init();
QMI8658_init();
xSemaphoreGive(i2c_mutex);
printf("Sensors Init OK\r\n");


// Create queues
xQueueButton = xQueueCreate(1, sizeof(bool));
xQueueSHTC3 = xQueueCreate(1, sizeof(SHTC3_data_t));
xQueueSGP40 = xQueueCreate(1, sizeof(uint32_t));
xQueueQMI8658 = xQueueCreate(1, sizeof(QMI8658_data_t));
xQueuePot = xQueueCreate(1, sizeof(uint16_t));
xQueueLight = xQueueCreate(1, sizeof(uint16_t));

```

```

xQueueSound = xQueueCreate(1, sizeof(uint16_t)); xQueueIR =
xQueueCreate(4, sizeof(IR_command_t));

// Check if all queues were created successfully
if (!xQueueButton || !xQueueSHTC3 || !xQueueSGP40 || !xQueueQMI8658 ||
    !xQueuePot || !xQueueLight || !xQueueSound || !xQueueIR) {
    printf("Failed to create queues\r\n");
    while(1);
}

// Create tasks
xTaskCreate(vButtonTask, "ButtonTask", 256, NULL, 1, NULL);
xTaskCreate(vSHTC3Task, "SHTC3Task", 256, NULL, 1, NULL);
xTaskCreate(vSGP40Task, "SGP40Task", 256, NULL, 1, NULL);
xTaskCreate(vQMI8658Task, "QMI8658Task", 256, NULL, 1, NULL);
xTaskCreate(vPotTask, "PotTask", 256, NULL, 1, NULL);
xTaskCreate(vLightTask, "LightTask", 256, NULL, 1, NULL);
xTaskCreate(vSoundTask, "SoundTask", 256, NULL, 1, NULL);
xTaskCreate(vIRTask, "IRTask", 256, NULL, 3, NULL);
xTaskCreate(vDisplayTask, "DisplayTask", 512, NULL, 1, NULL);

printf("All tasks created, starting scheduler\r\n");

// Start scheduler
vTaskStartScheduler();

// This code should never be reached while
(1) {
    // NOP
}

```

Висновок: У роботі було створено повноцінну багатозадачну систему FreeRTOS із підтримкою ІЧ-пульта за протоколом NEC, сенсорів, аналогових датчиків та OLED-дисплея. Дані від усіх задач передавалися через черги, а доступ до І²С захищався м'ютексом, що забезпечує коректну взаємодію сенсорів. Дисплей отримував дані з різних джерел та перемикав режими роботи відповідно до команд пульта. Система працює в реальному часі та адекватно реагує на дії користувача, забезпечуючи стабільне та передбачуване керування.